

CLIENT DELIVERABLE

Workflow Walkthrough

Systack Documentation

ROLE	COLOR	HEX
Headers, CTAs	Navy	 #001a2d
Navy Light	Navy Light	 #002845
Secondary accents	Teal	 #007da9
Primary buttons, links	Cyan	 #00a1db
Gradients	Cyan Bright	 #00c5e0
Backgrounds	Gray 50	 #f8fafc
Cards	Gray 100	 #f1f5f9
Borders	Gray 200	 #e2e8f0
Muted text	Gray 400	 #94a3b8
Body text	Gray 600	 #475569
Headings	Gray 800	 #1e293b
Success	Green	 #22c55e
Error	Red	 #ef4444
Accent highlights	Purple	 #8b5cf6

Utopia Deli — Confirmation Email System

Workflow Walkthrough

Document ID: UD-CONF-WF-001

Version: 1.0

Status: Internal — Systack Only

Source System: Utopia Deli (Live since 2026-06-12)

Date: 2026-06-29

Builder: SOL / Assembly

Brand Reference

ROLE	COLOR	HEX
Primary	Deep Burgundy	#590B3F
Primary Light	Burgundy Light	#7a1a55

ROLE	COLOR	HEX
Accent	Rust Red	#AF3D4B
Accent Hover	Rust Light	#c44d5b
Secondary	Purple	#754681
Gold	Warm Gold	#D59F5C
Gold Light	Cream	#f5e6d0
Background	Off-White	#FBFCFE
Card	White	#FFFFFF
Text	Dark Gray	#1F2937
Text Light	Medium Gray	#6B7280
Border	Light Gray	#E5E7EB
Success	Green	#22c55e
Error	Red	#dc2626

Workflow: Confirmation Email Dispatch

Workflow File: `utopia-confirmation-email-v3.json`

Node 1 — Webhook (Dual Trigger)

Type: Webhook (POST)

Accepts from two sources simultaneously (no merge node):

- **Frontend success page:** `POST /webhook/utopia-confirmation-email`
- **Square webhook:** `POST /webhook/utopia-square-webhook`

Frontend sends Square-compatible payload to unify processing:

```
{
  "type": "payment.updated",
  "data": {
    "object": {
      "payment": {
        "id": "frontend_UD0-xxx",
        "status": "COMPLETED",
        "reference_id": "UD0-xxx"
      }
    }
  }
}
```

Node 2 — Normalize

Type: Code Node

Parses both trigger formats:

- Extracts `reference_id` as `order_id`
 - Checks payment status is `COMPLETED`
 - Sets `process` flag (true/false)
-

Node 3 — Should Process?

Type: IF Node

Condition: `$json.process === true`

- **TRUE:** Continue to DB lookup
 - **FALSE:** Return “skipped” response
-

Node 4 — Prep DB Lookup

Type: Code Node

Prepares query parameters for SQLite lookup.

Node 5 — Lookup Order in DB

Type: SQLite Node

Queries `orders` table by `order_id`.

Node 6 — Extract DB Row

Type: Code Node

Critical fix: SQLite returns array `[{row}]`. This node extracts first row as object.

```
const rows = $json;
if (!rows || rows.length === 0) {
  return [{ json: { error: "NOT_FOUND" } }];
}
return [{ json: rows[0] }];
```

Node 7 — Order Exists?

Type: IF Node

Condition: `$json.error === "NOT_FOUND"`

- **TRUE:** → Not Found Response → Respond (404)
 - **FALSE:** → Continue to email check
-

Node 8 — Email Not Sent?

Type: IF Node

Condition: `Number($json.email_sent || 0) !== 1`

Prevents duplicate receipts:

EMAIL_SENT	RESULT
0 (default)	✔ Send email
1 (already sent)	⏏ Skip
null/undefined	✔ Send (Number(null) = 0)

- **TRUE:** → Build and send email
- **FALSE:** → Already Sent Response → Respond

Node 9 — Build Order Data

Type: Code Node

Parses `cart_json` from database into structured order data for the email template.

Node 10 — Build Cart HTML

Type: Code Node

Generates HTML table from cart items:

- Item name
- Quantity
- Unit price
- Line total

Node 11 — Build Branded Email

Type: Code Node

Constructs full HTML email:

- Utopia Deli burgundy header (`#590B3F`)
- “Payment received ✔ ” confirmation
- Order ID
- Itemized cart table
- Subtotal, tax, total
- Footer with address and phone

Node 12 — Email Exists?

Type: IF Node

Condition: `$json.email !== "" && $json.email !== null && $json.email !== undefined`

Guards against sending to empty addresses.

- **TRUE:** → Send Email
- **FALSE:** → No Email Response → Respond

Node 13 — Send Email

Type: Email Send (SMTP)

Sends via Gmail SMTP:

- **To:** Customer email
- **Subject:** Your Utopia Deli Order Receipt
- **Body:** Branded HTML template

Node 14 — Mark Email Sent

Type: SQLite Node

Updates database:

```
UPDATE orders SET email_sent = 1, email_sent_at = datetime('now') WHERE order_id = ?
```

Node 15 — Respond

Type: Respond to Webhook

Returns JSON confirmation:

```
{
  "success": true,
  "order_id": "UD0-xxx",
  "email_sent": true
}
```

Complete Node Chain

```
Webhook (Square or Frontend)
  → Normalize
  → Should Process? (IF)
    → Prep DB Lookup
      → Lookup Order in DB (SQLite)
        → Extract DB Row
          → Order Exists? (IF)
            → Email Not Sent? (IF)
              → Build Order Data
                → Build Cart HTML
```

```
→ Build Branded Email
  → Email Exists? (IF)
    → Send Email (SMTP)
      → Mark Email Sent (SQLite)
        → Respond ✅
```

Error Branches

BRANCH	CONDITION	RESPONSE
Not Processed	Payment status ≠ COMPLETED	<code>{"skipped": true}</code>
Not Found	Order ID not in DB	<code>{"error": "NOT_FOUND"}</code>
Already Sent	email_sent = 1	<code>{"already_sent": true}</code>
No Email	Customer email empty	<code>{"error": "NO_EMAIL"}</code>

Internal document — do not share with clients.

© 2026 Systack. All rights reserved.